

DAY 12 – ADVANCED STRING CONCEPTS & PATTERN PROBLEMS

Mastery of DSA in 50 Days

February 2026

1 Today's Focus

- Strengthening string problem-solving skills
- Understanding pattern-based string problems
- Mastering frequency counting techniques
- Optimizing string logic with hashing / arrays
- Developing time complexity awareness

2 What are Pattern-based String Problems?

Pattern-based string problems typically involve:

- Repeating / non-repeating characters
- Character frequency comparison
- Matching conditions across strings
- Structured traversal with constraints

These problems heavily rely on:

- String traversal
- Frequency counting (array or hashmap)
- Multiple-pass logic
- Avoiding nested loops

3 Problem 1 – Count Frequency of Characters

Problem Statement: Given a string, count the frequency of each character and print them.

Example:

Input: "aabbbc"

Output:

a → 2

b → 3

c → 1

Logic (using array for lowercase letters):

```

1 def count_frequency(s):
2     # Assuming lowercase English letters
3     freq = [0] * 26
4
5     for char in s:
6         if char.isalpha():
7             idx = ord(char) - ord('a')
8             freq[idx] += 1
9
10    for i in range(26):
11        if freq[i] > 0:
12            print(chr(i + ord('a')), " ", freq[i])
13
14 # Test
15 count_frequency("aabbbc")
16 # a      2
17 # b      3
18 # c      1

```

Time Complexity: $O(n)$ Space Complexity: $O(1)$ (fixed 26 letters)

Alternative: Use dict / Counter for any characters $\rightarrow O(n)$ time, $O(k)$ space (k = unique chars)

4 Problem 2 – Check if Two Strings are Anagrams

Problem Statement: Check whether two strings are anagrams (same characters with same frequency, different order).

Example:

Input: `s1 = "listen", s2 = "silent"`

Output: `True`

Optimal Logic (Frequency Count):

```

1 from collections import Counter
2
3 def is_anagram(s1, s2):
4     if len(s1) != len(s2):
5         return False
6
7     # Method 1: Using Counter (clean & readable)
8     return Counter(s1) == Counter(s2)
9
10    # Method 2: Using array (faster for lowercase only)
11    # freq = [0] * 26
12    # for c in s1: freq[ord(c) - ord('a')] += 1
13    # for c in s2: freq[ord(c) - ord('a')] -= 1
14    # return all(x == 0 for x in freq)
15
16 # Test
17 print(is_anagram("listen", "silent")) # True
18 print(is_anagram("hello", "world"))  # False

```

Time Complexity: $O(n)$ Space Complexity: $O(1)$ (array) or $O(k)$ (hashmap)

5 Problem 3 – First Non-Repeating Character

Problem Statement: Find the index of the first character that appears only once in the string. Return -1 if no such character exists.

Example:

Input: "aabbcdde"

Output: 4 (index of 'c')

Logic (Two-pass approach):

```

1 def first_non_repeating(s):
2     if not s:
3         return -1
4
5     # Step 1: Count frequency
6     freq = [0] * 256 # ASCII range
7     for char in s:
8         freq[ord(char)] += 1
9
10    # Step 2: Find first char with freq == 1
11    for i in range(len(s)):
12        if freq[ord(s[i])] == 1:
13            return i
14
15    return -1
16
17 # Test
18 print(first_non_repeating("aabbcdde")) # 4 ('c')
19 print(first_non_repeating("aabbcc"))  # -1

```

Time Complexity: $O(n)$ Space Complexity: $O(1)$ (fixed 256 ASCII)

6 Key String Techniques Used Today

- Single-pass / two-pass traversal
- Frequency counting (array for fixed charset, hashmap for general)
- Comparing frequency maps
- Index tracking for first occurrence
- Avoiding nested loops

7 Common Mistakes to Avoid

- Forgetting to check string lengths (anagram)
- Not handling uppercase/lowercase differences
- Using nested loops for frequency comparison
- Assuming all characters are lowercase letters
- Returning wrong index/value when no solution exists

8 Key Learnings – Day 12

- Many string problems reduce to frequency counting
- Frequency array is $O(1)$ space when alphabet is small (26/256)
- Two-pass approach is very common: count $\rightarrow$ decide
- Always aim for $O(n)$ time $\rightarrow$ avoid $O(n^2)$ brute force
- Hashing / array choice depends on character set size

9 Interview Tips

- When you see “count”, “frequency”, “same characters”, “non-repeating” $\rightarrow$ think frequency counting
- Ask: “What is the character set? (lowercase only? ASCII? Unicode?)”
- Prefer array over hashmap when possible (faster & constant space)
- State complexity clearly: “ $O(n)$ time, $O(1)$ space using frequency array”
- Handle edge cases: empty string, single char, all repeating

10 Note to Learners

Frequency-based string problems appear in almost every interview round.

Mastering them gives you a huge advantage in:

- Anagram / permutation problems
- Longest substring without repeating characters
- Group anagrams
- Minimum window substring
- Many medium LeetCode problems

End of Day 12

Next: Day 13 – Hashing Basics